

PRACTICAL-WEEK3

Task-1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Source code:

```
GNU nano 8.7.1 main.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child is running...\n");

        sleep(5);

        printf("Child process completed.\n");
        printf("Child exiting...\n");

        exit(0);
    }
}
```

Output:

```
saichaitanya@GeethzzSai:~$ cd practical_week3
saichaitanya@GeethzzSai:~/practical_week3$ nano main.c
saichaitanya@GeethzzSai:~/practical_week3$ gcc main.c -o main
saichaitanya@GeethzzSai:~/practical_week3$ ./main
Parent process started
Parent PID : 725
Parent PPID : 645

--- PARENT PROCESS ---
Parent PID : 725
Child PID : 726
Parent is waiting for child...

--- CHILD PROCESS ---
Child PID : 726
Child PPID : 725
Child is running...
Child process completed.
Child exiting...
Child process has terminated.
Parent process completed.
```

Observation: I learned how the fork() system call creates a child process from a parent process. I understood how to display and differentiate the PID and PPID of parent and child processes. I also learned how a parent process waits for the child process to complete using wait() and observed the different process states during execution

Task2: The aim is to observe process state transitions such as Running, Waiting/Sleeping, and Terminated using Linux commands like ps, top, and /proc.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid;

    printf("=== PROCESS CREATION ===\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());

    printf("\nParent is RUNNING before fork()\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {

        /* CHILD PROCESS */

        printf("\n=== CHILD PROCESS ===\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child state: RUNNING\n");

        printf("Child will sleep for 10 seconds...\n");

        sleep(10);

        printf("Child is RUNNING again after sleep.\n");

        printf("Child is TERMINATING.\n");

        exit(0);
    }

    else {

        /* PARENT PROCESS */

        printf("\n=== PARENT PROCESS ===\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent state: WAITING\n");
        printf("Parent is waiting for child...\n");

        waitpid(pid, NULL, 0);

        printf("\nChild has TERMINATED.\n");

        printf("Parent is RUNNING again.\n");

        printf("Parent will terminate.\n");
    }
}
```

Output:

```
top - 03:38:10 up 46 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7782.6 total, 7195.6 free, 516.6 used, 220.4 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,  0.0 used. 7266.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	21592	13220	9796	S	0.0	0.2	0:00.50	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.00	init-systemd(Ub
7	root	20	0	3196	2132	2008	S	0.0	0.0	0:00.00	init
53	root	19	-1	33976	12636	11560	S	0.0	0.2	0:00.18	systemd-journal
103	root	20	0	25464	6852	5124	S	0.0	0.1	0:00.55	systemd-udevd
112	systemd+	20	0	21344	13184	11056	S	0.0	0.2	0:00.07	systemd-resolve
120	systemd+	20	0	91036	7988	7020	S	0.0	0.1	0:00.09	systemd-timesyn
170	root	20	0	4244	2684	2432	S	0.0	0.0	0:00.01	cron
171	message+	20	0	9528	5364	4668	S	0.0	0.1	0:00.06	dbus-daemon
188	root	20	0	17980	8852	7808	S	0.0	0.1	0:00.09	systemd-logind
190	root	20	0	1755848	10780	8404	S	0.0	0.1	0:00.16	wsl-pro-service
209	syslog	20	0	222516	6068	4676	S	0.0	0.1	0:00.08	rsyslogd
212	root	20	0	3124	1972	1836	S	0.0	0.0	0:00.00	agetty
218	root	20	0	107032	23184	13724	S	0.0	0.3	0:00.12	unattended-upgr
296	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
297	root	20	0	3200	1244	1108	S	0.0	0.0	0:00.14	Relay(301)
301	root	20	0	6188	5464	3708	S	0.0	0.1	0:00.14	bash
304	root	20	0	6672	4496	3728	S	0.0	0.1	0:00.00	login
351	root	20	0	20328	11512	9388	S	0.0	0.1	0:00.04	systemd
352	root	20	0	21168	3496	1784	S	0.0	0.0	0:00.00	(sd-pam)
372	root	20	0	6080	5332	3668	S	0.0	0.1	0:00.01	bash
815	root	20	0	9280	5660	3500	R	0.0	0.1	0:00.02	top

```
saichaitanya@GeethzzSai:~/practical_week3$ gcc main2.c -o main2
saichaitanya@GeethzzSai:~/practical_week3$ ./main2
=== PROCESS CREATION ===
Parent PID   : 793
Parent PPID  : 645

Parent is RUNNING before fork()

=== PARENT PROCESS ===
Parent PID   : 793
Child PID    : 794
Parent state: WAITING
Parent is waiting for child...

=== CHILD PROCESS ===
Child PID    : 794
Child PPID   : 793
Child state: RUNNING
Child will sleep for 10 seconds...
Child is RUNNING again after sleep.
Child is TERMINATING.

Child has TERMINATED.
Parent is RUNNING again.
Parent will terminate.
```

